

Differences between Hyperledger Fabric and BigchainDB

BigchainDB is a blockchain based database that is decentralised, queryable, immutable, has native support or multiple assets, is Byzantine fault tolerant and much more. However, BigchainDB being a blockchain database, lacks the processing layer or the business logic (smart contract) layer for the assets it stores. While we have platforms like Hyperledger Fabric, which excel at providing smart contract services, they are not platforms that provide user friendly querying functionality the way BigchainDB does.

So, for effectively creating the smart contracts for use across industries, integrating Hyperledger Fabric (which provides the business logic layer) and BigchainDB (which provides excellent functionalities of a database and the distributed ledger technology) provides an incredible solution for the digital future.

Table 1 highlights the differences between Hyperledger Fabric and BigchainDB.

Overall, with tabular analysis, Hyperledger Fabric is suitable for complex business cases whereas BigchainDB provides basic functionalities and mostly can be used in financial organisations.

Differences between Bitcoin, distributed databases and BigchainDB

Table 2 highlights the differences between Bitcoin, distributed databases and BigchainDB.

Table 2

Point of difference	Bitcoin	Distributed database	BigchainDB
Centralised authority	No	Yes	No
Immutability	Yes	No	Yes
Assets over the network	Yes	No	Yes
Best throughput	No	Yes	Yes
Low latency	No	Yes	Yes
Access control with high security features	No	Yes	Yes
Best queries to data-base—flexible control	No	Average	Yes

Configuring the BigChainDB network

To configure the network, you first need to deploy a machine for the BigchainDB node. The requirements for deployment are listed below.

1. This can be any standalone machine, virtual machine, VM running on a cloud server or even a Linux based computer like Raspberry Pi.
2. All nodes in the network should have a public or private IP address.
3. Ubuntu 18.04/19.04/19.10 server operating system.

Here are the steps you need to follow for configuration.

Step 1: Update the system using the following code:

```
sudo apt update
sudo apt full-upgrade
```

Step 2: DNS setup:

- a. Register a domain for the BigchainDB node, i.e., *server.com*.
- b. Add a sub-domain for BigchainDB node, i.e., *node1.server.com*.
- c. Create a DNS 'A RECORD' for *node1.server.com* at your machine IP address.

Step 3: Install NGINX as follows:

```
sudo apt update
sudo apt install nginx
```

Step 4: Configure and reload NGINX.

- a. Generate an SSL certificate for the node subdomain, i.e., *node1.server.com*.

- b. Copy the SSL private key into */etc/nginx/ssl/cert.key*.
- c. Create a 'PEM File' by concatenating your SSL certificate with all intermediate certificates.
- d. Copy that PEM file into */etc/nginx/ssl/cert.pem*.
- e. Under the *Bigchaindb/bigchaindb* repository on GitHub, locate the file *nginx/nginx.conf* and copy the contents into */etc/nginx/nginx.conf* on the machine.
- f. Edit the file */etc/nginx/nginx.conf* and relate the two instances of the string *example.testnet2.com* with the name of the sub-domain, i.e., *node1.server.com*.
- g. Reload NGINX – *sudo service nginx reload*.

Step 5: Install BigchainDB, MongoDB and Tendermint.

- a. Make sure that Python 3.6+ is installed in the machine and install the other OS-level packages.

```
sudo apt install -y python3-pip libssl-dev
```

- b. BigchainDB server requires *gevent* and to install *gevent*, Pip19 or later must be installed.

```
sudo pip3 install -U pip
```

- c. Now install BigchainDB server:

```
sudo pip3 install bigchaindb==2.0.0b9
```

Step 6: Configure the BigchainDB server.

- a. Run the following command:

```
bigchaindb configure
```

The first question is: API Server Bind? (default 'localhost:9984').

If you are using NGINX, then accept the default value. And if you are not using NGINX, then enter: 0.0.0.0:9984.

If you are using NGINX, edit the BigchainDB config file--*\$home/.bigchaindb* and set the following values under "wsserver":